

CANDY DIALOGUE ENGINE

Dialogue History

1.0.0

Saving History

Candy DE's Dialogue History feature saves a history of previous spoken dialogues, in order to be displayed to the player later.

The saving of dialogue history occurs automatically, provided that `write_dialogue_history` is set to 'true' in [Candy_Engine.gd](#).

History data is saved in the `dialogue_history` array in [Candy_Engine.gd](#) and in `general_dialogue_history` in [Candy_Database.gd](#). The reason it is saved in both places is in case you run multiple dialogue engine instances: `dialogue_history` stores only the history of its own instance, while `general_dialogue_history` stores the history of all instances.

Note that by default, only spoken lines and transitions are saved. Any other commands are not saved, as they generally don't provide information the player might find relevant for story purposes.

If you wish to save other commands to the `dialogue_history` array, open [Candy_Engine.gd](#), and go to the `process_line()` function at Step 10. There, add the functions you desire under the 'if last_cm in [...]' statement, following the same format as the commands already listed:

```
## Step 10 - Dialogue history logging:
if write_dialogue_history == true:
    if last_cmd in [command_symbol + "jump", command_symbol + "bridge", command_symbol + "end", command_symbol + "return"]:
        var clean_name = last_cmd.substr(command_symbol.length()).capitalize()

        if clean_name == "End":
            dialogue_history.append({"End": ""})
            candy_de.general_dialogue_history.append({"End": ""})
        else:
            var new_conv: String = str(line_data.get("Conversation", "")).strip_edges()
            var same_conv = (new_conv == "" or new_conv == current_conversation)
            dialogue_history.append({clean_name: { "changed_conv": not same_conv }})
            candy_de.general_dialogue_history.append({clean_name: { "changed_conv": not same_conv }})
```

For Spoken Lines, you'll need to navigate to the `display_line()` function. Under each dialogue style (or at least the styles you use), you'll need to change the code block titled '#@ Dialogue writing speed logic'.

```

#@ Dialogue writing speed logic:
var total_visible = bbcode_strip_tags(bbcode_text).length()
dialogue_box.caller = self
if writing_speed <= 0:
    dialogue_text_node.text = bbcode_text
    await wait_for_player_advance()
    if vn_mode == true and bust_positions.has(speaker_ref):
        get_node(ui_elements_paths["busts_path"]).end_highlight_speaker(speaker_ref, style)
    dialogue_history.append({
        "Speech": {
            "speaker_ref": speaker_ref,
            "display_name": speaker_display,
            "speech": dialogue_text,
            "portrait": portrait_raw,
            "disposition": disposition
        }
    })
candy_de.general_dialogue_history.append({
    "Speech": {
        "speaker_ref": speaker_ref,
        "display_name": speaker_display,
        "speech": dialogue_text,
        "portrait": portrait_raw,
        "disposition": disposition
    }
})

```

Specifically, you'll want to modify the following code:

```
dialogue_history.append({})
candy_de.general_dialogue_history.append({})
```

Change it to append whatever data you want: you can later define the logic that reads both arrays and displays dialogue history.

Displaying History

Opening History

In order to display dialogue history to the player, you will need to call the `open_instance_history()` function or the `open_general_history()` function inside `Candy_Functions.gd`.

```

/* Dialogue engine instance history:
  #% Make sure the correct dialogue engine instance is passed as 'caller' when the function is called.
  func open_instance_history(caller):
      var history_ui = get_node(caller.history_display_path)
      history_ui.caller = caller
      history_ui.show_history()

  /* General history:
  #% Call this from anywhere.
  func open_general_history():
      var history_ui = get_node("")    #TODO: Add the path to your scene that displays the general history
      history_ui.caller = null
      history_ui.show_history()

```

You can call these functions any way you like: make them open automatically at set points during the game, when the player clicks a button, or when the player presses a key. Remember that you can call them from anywhere by prefixing them with '**candy_de**'.

The **open_instance_history()** function accepts an argument, 'caller', which should be the dialogue engine instance you want to display the history of. By default, just pass '**candy_ui**' (no quotation marks) as the value for 'caller': this will display the history log of the Candy_UI.

History Scene

A basic History Scene is provided by default in **Candy_DE/Scenes/History Menus**, but like all UI elements, you can create your own.

You can place it any where you want in your game's node hierarchy. Dialogue engine instances can share the same History UI, or can each have their own.

Just be sure to do the following:

For each dialogue engine instance, in the **Candy_Engine.gd** script, indicate the path to the History UI that the given instance should use, in **ui_elements_paths**.

In the **open_general_history()** function, provide the path to the History UI that should be used, in the **history_ui** variable:

```
var history_ui = get_node("")
```

Display

When displayed, the dialogue history is automatically formatted for readability. By default, speaker names and spoken text are displayed side by side, like this:

Alice: Hello!

Transitions are represented by a BBCode horizontal line, but only when a transition was made to another Conversation. Transitions to different Blocks or lines in the same Conversation are not displayed.

Other commands are not displayed, unless you modify the code to add them to the history log and display them.

Finally, wherever dialogue ended, a series of asterisks is displayed.

You can modify the **update_history_display()** function in the **History_Display.gd** script to change this formatting, which should be easy enough even if you are a beginner in Godot or programming altogether.

Other Uses

The **dialogue_history** array of each engine instance, and the **general_dialogue_history** array (which store the history log) can be used for other purposes besides displaying dialogue history to

the player, such as debugging or retrieving expanded dialogue context if you use LLM-generated dialogue.

There are no built-in features for these uses, as this is heavily dependent on your own needs. However, writing such code should be fairly simple.